

# ТЕМА: ДЕРЕВЬЯ



## ВВЕДЕНИЕ

Деревья. Казалось бы - все просто: есть корень, есть лист и на этом все. Но это заблуждение, в которое вам не стоит впадать. В этой теме вы узнаете все что требуется для работы с деревьями

## УРОК: ОБХОД В ГЛУБИНУ

### Обход в глубину DFS

это метод обхода дерева или графа, при котором сначала посещаются все узлы в глубину, начиная с текущего узла, прежде чем переходить к соседним узлам



### Принципы работы

- 01 Используется стек (или рекурсия) для хранения узлов, которые должны быть посещены
- 02 Начните с корня и добавьте его в стек
- 03 Пока стек не пуст
- 04 Извлеките узел из стека
- 05 Посетите его и обработайте данные
- 06 Добавьте всех его детей в стек

## Типы обхода в глубину

### Обход по предзаказу Pre-order

Посещение корня, затем левого поддерева, затем правого поддерева

### Обход по порядку In-order

Посещение левого поддерева, корня, затем правого поддерева

### Обход по постзаказу Post-order

Посещение левого поддерева, правого поддерева, затем корня

## Пример реализации обхода в глубину

```
def dfs_preorder(node):
    if node:
        print(node.value, end=' ')
        dfs_preorder(node.left)
        dfs_preorder(node.right)

def dfs_inorder(node):
    if node:
        dfs_inorder(node.left)
        print(node.value, end=' ')
        dfs_inorder(node.right)

def dfs_postorder(node):
    if node:
        dfs_postorder(node.left)
        dfs_postorder(node.right)
        print(node.value, end=' ')

# Пример использования:
dfs_preorder(bt.root) # Вывод: 5 3 7
dfs_inorder(bt.root)  # Вывод: 3 5 7
dfs_postorder(bt.root) # Вывод: 3 7 5
```

## Преимущества и недостатки

### Преимущества

Подходит для задач, где важно исследовать все возможные пути

### Недостатки

Может потреблять много памяти в случае глубоких деревьев, особенно при использовании рекурсии

## ВЫВОД

Мы рассмотрели метод обхода в глубину. Понимание этой концепции поможет вам эффективно управлять и обрабатывать данные в различных приложениях и задачах